# ZAP-MCP and ZAP-A2A: Mutually Authenticated, Non-Repudiable Transport for the MCP and Agent2Agent Protocols

ZAP Protocol Authors
zap-proto.io

2026

**Abstract**

Autonomous and semi-autonomous AI agents have moved from research demos to infrastructure-critical systems: they execute trades, retrieve patient records, and modify production code. The two protocols that knit these agents together — *Model Context Protocol* (MCP) for agent-to-tool calls [17] and *Agent2Agent* (A2A) for agent-to-agent collaboration [1] — inherit their security posture from the legacy web stack: TLS for confidentiality, API keys or JWT bearers for authentication, and HTTP/SSE or HTTP/JSON framing. We argue that this stack is structurally inadequate for an agent fabric. Bearer tokens are bearer-authenticated, not message-authenticated; JSON-RPC payloads carry no identity; and intermediate agent gateways or tool brokers are forced to terminate TLS in order to route, breaking end-to-end confidentiality and provenance. We present ZAP-MCP and ZAP-A2A, two compositions of the ZAP post-quantum authenticated transport with the MCP and A2A application protocols. Each agent-to-tool or agent-to-agent message is wrapped in a ZAP frame, keyed to a Module-Lattice KEM public key, signed under a per-agent ML-DSA-65 (or hybrid Ed25519+ML-DSA-65) signing key, and bound to a session-scoped nonce ladder. We give security definitions, prove non-repudiation under the EUF-CMA assumption for ML-DSA, prove agent-impersonation infeasibility under combined KEM IND-CCA and signature EUF-CMA assumptions, and bound replay advantage by the nonce-collision probability. We also analyse wire overhead and show that ZAP-wrapped MCP/A2A traffic is shorter on the wire than HTTP+SSE+JWT baselines for typical tool-call sizes, while delivering audit-grade provenance suitable for HIPAA, SOC 2, and GDPR Article 30 records of processing.

## 1 Introduction

### 1.1 Agents are now critical infrastructure

A 2023 LLM-integrated chatbot is recoverable; a 2026 agent that has funded your accounts, called your APIs, and rewritten your production code is not. Agents are increasingly given tools that perform irreversible actions: order routing, database mutations, identity issuance, code deployment, fund transfers, prescription writing. The protocols that carry these actions — MCP for tool invocation and A2A for inter-agent collaboration — are the new universal joints of automated labor.

These protocols were specified for fast adoption, not for long-horizon auditability or post-quantum survivability. MCP runs JSON-RPC 2.0 over stdio (no auth), HTTP+SSE (auth via TLS + an opaque "API key" header or OAuth bearer token), or WebSocket (likewise) [14, 17]. A2A runs structured tasks and artifacts over HTTP/JSON, with agent identification expressed via "agent cards" typically signed only by the TLS certificate of the host [1]. In both cases, the security boundary is the TLS connection, not the message.

### 1.2 Why the TLS-and-bearer stack is inadequate

We identify five structural gaps:

1. **Tool impersonation.** A bearer token attests that the *client* has presented credentials to *some* server reached at a URL. It does not attest that the server is the intended tool. URL phishing or DNS substitution succeeds whenever the agent has not out-of-band pinned the tool's certificate.
2. **Message non-repudiation.** TLS protects messages *in flight*, not afterwards. A logged JSON-RPC envelope cannot later be verified to have actually been emitted by a given agent: TLS authentication is symmetric session-keyed and unprovable to a third party. Disputes over "did agent $A$ really request to wire \$10 M?" have no cryptographic answer.
3. **Cross-session replay.** A bearer-token-authenticated request captured at runtime can be replayed by an attacker with the same token, because the request itself is not bound to a fresh session secret.
4. **Quantum-vulnerable transcripts.** Long-lived agent logs (medical, legal, financial) recorded today and decryptable in 2035 by an attacker holding a CRQC are out of policy under "harvest-now-decrypt-later" [18] for any organisation whose data retention exceeds the quantum horizon.
5. **Broker-opaque routing.** Gateways that route between agent clouds must today terminate TLS in order to read MCP/A2A message envelopes; this destroys end-to-end secrecy and forces gateways into the trusted compute base.

### 1.3 Thesis

Wrapping MCP and A2A in ZAP gives every agent-to-tool and agent-to-agent message: post-quantum confidentiality, mutual cryptographic identity, non-repudiation, and provenance — all at lower wire overhead than the current HTTP+SSE / HTTP+JWT defaults. Concretely:

- Each MCP JSON-RPC request and response, and each A2A task or artifact, is encapsulated as a single ZAP frame.
- Each frame carries an ML-DSA-65 signature of the request hash by the agent's long-term signing key (or a hybrid Ed25519+ML-DSA-65 pair during transition).
- Each frame is encrypted to the recipient's ML-KEM-768 public key via an X-Wing-style hybrid construction [5, 19].
- The session is initiated by a ZAP 1-RTT handshake whose transcript is signed by both parties; thereafter, frames carry a sequence-bound nonce.
- Agent identity is resolved through ZAP-RNS [30]: a name maps to an ML-KEM public key and a signing public key, replacing the "URL plus host certificate" identity model.

The remainder of this paper formalises the construction (Sections 3–5), states and proves non-repudiation, replay resistance, and impersonation resistance (Section 6), measures wire overhead (Section 7), and discusses compliance, deployment, and gap items (Sections 9–11).

## 2 Background

### 2.1 Model Context Protocol

MCP [17] is a JSON-RPC 2.0 [14] request/response protocol between an *agent* (client, typically driven by an LLM) and a *tool server*. The agent invokes methods such as `tools/list`,

`tools/call`, `resources/read`, and `prompts/get`; the tool server returns structured results. Three transports are specified: stdio (in-process), HTTP+SSE (request by HTTP POST, asynchronous server messages by Server-Sent Events [9]), and WebSocket. Authentication is undefined at the protocol level; in practice, deployments inject an `Authorization` header containing an OAuth 2.0 bearer token [11, 12].

### 2.2 Agent2Agent

A2A [1] is a higher-level protocol over HTTP/JSON. Agents publish *agent cards* describing their capabilities; clients invoke long-running *tasks*, which produce *artifacts* consumed by other agents or terminating in a final result. A2A authentication is HTTP-layer, typically TLS server authentication plus a bearer JWT.

### 2.3 The ZAP transport

ZAP [29] is a binary, stateful, post-quantum authenticated transport. A ZAP session is established by a 1-RTT handshake derived from the SIGMA [15] family, parameterised by a hybrid KEM (ML-KEM-768 with X25519 [5, 19]) and a hybrid signature (Ed25519 with ML-DSA-65 [4, 20]). Frames carry a 64-bit sequence number, a 96-bit nonce derived from HKDF [16] of the session key, and an authenticated ciphertext under AES-256-GCM or ChaCha20-Poly1305. Optionally a *signed-payload* flag adds a per-frame ML-DSA-65 signature over the plaintext payload hash, providing third-party verifiable non-repudiation.

### 2.4 ZAP-RNS: identity-bound naming

ZAP-RNS [30] resolves a human-readable agent or tool name to a record containing (i) an ML-KEM public key, (ii) an Ed25519 public key, (iii) an ML-DSA-65 public key, (iv) a vector of capability claims, and (v) a signature chain rooted at a publishing authority. Resolution returns a self-authenticating record; identity is no longer expressed as "hostname + a certificate someone in the chain trusts".

## 3 Threat Model

### 3.1 Trust assumptions

**Trusted:** the agent's long-term signing key and the tool's long-term decapsulation key (each held in an HSM or equivalent isolated enclave); the ZAP-RNS record cache after first authenticated fetch; the cryptographic primitives ML-DSA-65, ML-KEM-768, AES-GCM, SHA-3, HKDF.

**Untrusted:** the network; any intermediate "agent gateway" performing routing or load balancing; the JSON-RPC transcript on disk (must be third-party verifiable); the DNS resolver (because identity binds to KEM PK, not DNS).

### 3.2 Adversary capabilities

We assume an active network adversary $\mathcal{A}$ who:

- observes, drops, reorders, modifies, and injects all network traffic;
- operates a polynomial number of legitimate-looking tool servers and agents and can cause them to engage in handshakes;
- may corrupt up to all but one of the parties in the system, provided we explicitly declare the surviving party honest;

- is bounded to probabilistic polynomial time, including polynomially many quantum oracle queries (the assumption used by ML-KEM-768 and ML-DSA-65 security claims [19, 20]);
- may store transcripts indefinitely (harvest-now-decrypt-later).

### 3.3 Concrete agent-fabric attacks

We consider the following attacks of practical importance, and the section that addresses each:

1. *Tool impersonation.* $\mathcal{A}$ stands up a malicious tool server claiming to be a legitimate API. **Addressed by** ZAP-RNS-bound mutual authentication (§4).
2. *Indirect prompt injection via tool output* [10]. Out of scope cryptographically (prompt injection lives at the model layer); however, ZAP provides the prerequisite that tool output is cryptographically attributable to a named tool, so injection-induced damage is forensically reconstructible.
3. *Cross-agent replay.* $\mathcal{A}$ captures a tool call from agent $A$ and replays it as if from $A$, on a different session. **Addressed by** session-bound nonces (Lemma 7 in [31]).
4. *Long-term provenance loss.* Six months later, agent $A$ denies having issued a transaction. **Addressed by** ML-DSA-65 signed frames (Theorem 5).
5. *Future-quantum decryption* of medical/legal transcripts. **Addressed by** hybrid ML-KEM-768 + X25519 frame keying.
6. *Broker-introduced injection.* A routing gateway modifies MCP request fields. **Addressed by** per-frame AEAD: a routing gateway cannot decrypt or modify without invalidating the frame tag and signature.

## 4 ZAP-MCP Construction

### 4.1 Wire mapping

A MCP JSON-RPC request is a JSON object $\rho = \{\texttt{jsonrpc}, \texttt{id}, \texttt{method}, \texttt{params}\}$. Define the canonical encoding

$$\hat{\rho} = \mathrm{CBOR}(\rho)$$

using deterministic CBOR [23]. Let $\sigma = \mathsf{Sig.Sign}(sk_A, h(\hat{\rho} \parallel ctx))$ where $h$ is SHA3-256, $sk_A$ is agent $A$'s ML-DSA-65 signing key, and $ctx$ is a binding context string of the form

$$ctx = \text{"}\texttt{zap} - \texttt{mcp/v1}\text{"} \parallel sid \parallel seq$$

where $sid$ is the 32-byte session identifier produced by the ZAP handshake and $seq$ is the 64-bit frame sequence number.

The wrapped request is the ZAP frame

$$F = \mathsf{ZAP.Seal}(K_{AB},\ nonce(sid, seq),\ \hat{\rho} \parallel \sigma,\ ad)$$

where $K_{AB}$ is the per-direction AEAD key derived from the handshake, and $ad = \text{"}\texttt{zap} - \texttt{mcp/v1/req}\text{"} \parallel seq$.

The corresponding MCP response from tool $T$ is wrapped symmetrically under $T$'s signing key, with $ctx$ extended to bind to the request hash, ensuring response-to-request linkage.

### 4.2 Mutual identification

The agent locates the tool by name $n$ via ZAP-RNS: it retrieves a record $R_T$ binding $n$ to a tuple $(kem\_pk_T, sig\_pk_T, caps_T)$ with a publisher signature. The agent verifies the publisher chain, then initiates a ZAP handshake against the tool's network endpoint using $kem\_pk_T$. The handshake transcript is signed by both parties.

*Remark* 1. Crucially, the agent never trusts a hostname. If DNS misroutes the agent to an attacker's endpoint, the handshake fails because the attacker does not hold the secret key for $kem\_pk_T$. This eliminates the entire class of tool-URL phishing attacks.

### 4.3 Capability claims

ZAP-RNS records carry capability claims, e.g. "this tool is permitted to list orders but not place them". The handshake transcript binds the hash of the claims set; the tool must therefore present a record whose claims include the methods the agent will call. The agent can refuse `tools/call` for methods absent from the claim vector before any network round trip.

### 4.4 Pseudocode (request path)

```
function zapmcp_call(agent_sk, tool_name, method, params):
    R_T  := rns_resolve(tool_name)
    verify_record_chain(R_T)
    require method in R_T.caps

    sess := zap_handshake(R_T.kem_pk, agent_sk)  // 1-RTT
    rho  := { jsonrpc: "2.0", id: fresh_id(),
              method: method, params: params }
    rho_hat := cbor(rho)
    ctx  := "zap-mcp/v1" || sess.id || sess.next_seq()
    sig  := mldsa65_sign(agent_sk, sha3_256(rho_hat || ctx))
    F    := zap_seal(sess, rho_hat || sig)
    send(F)
    F'   := recv()
    payload, sig' := zap_open(sess, F')
    require mldsa65_verify(R_T.sig_pk,
                           sha3_256(payload || resp_ctx), sig')
    return cbor_decode(payload)
```

## 5 ZAP-A2A Construction

A2A is task-shaped, not call-shaped: an agent submits a task description, the remote agent acknowledges and emits a sequence of artifacts and progress events, and eventually a final result. We map each top-level A2A message — task envelope, artifact, status event, terminal result — to one signed ZAP frame, with the following augmentations.

### 5.1 Signed agent cards

In A2A an *agent card* declares an agent's identity, capabilities, and endpoints. Under ZAP-A2A, an agent card is a canonical CBOR object signed by the agent's ML-DSA-65 key and published through ZAP-RNS. A consuming agent verifies the card off the wire; the card is itself a frame-shaped, third-party-verifiable artifact.

### 5.2 Artifact provenance

Each artifact emitted during a task carries a frame signature by the producing agent. Because frames bind *sid* (session id) and the parent task identifier, an artifact is independently verifiable as "produced by agent $B$, in the context of task $\tau$, in session *sid*, at sequence *seq*". A verifier presented with only the artifact and the public keys of $B$ can confirm authenticity without access to the original session.

### 5.3 Multi-hop tasks

When agent $A$ delegates to $B$ which subdelegates to $C$, the chain of frames forms a directed acyclic graph of signed claims. Each edge is independently verifiable. ZAP-A2A does not require any single party to be online during audit. This is in deliberate contrast to the bearer-token model, where revocation of $B$'s token retroactively invalidates the chain.

### 5.4 Encrypted-to-recipient routing

A frame is encrypted under the recipient's per-session AEAD key, itself derived from the recipient's ML-KEM static key. A routing gateway sees only the routing header (sender, recipient ZAP-RNS identifiers, frame size, sequence). The gateway cannot read or modify content, removing an entire trusted compute base from the agent fabric.

## 6 Security

We give formal statements here. Full proofs are in the companion proof document [31].

### 6.1 Definitions

**Definition 2** (Agent non-repudiation)**.** For a frame $F$ produced in session *sid* at sequence *seq*, claimed sender $A$, payload $\hat{\rho}$, signature $\sigma$: $F$ is non-repudiable if any third-party verifier $V$ holding $sig\_pk_A$ can decide whether $\sigma$ is a valid ML-DSA-65 signature on $h(\hat{\rho} \,\|\, ctx)$, with binding context *ctx* as defined above. The non-repudiation advantage of $\mathcal{A}$ is the probability that $\mathcal{A}$ produces a triple $(F^*, A^*, \hat{\rho}^*)$ accepted by $V$ as authored by $A^*$ when in fact $A^*$ never signed it.

**Definition 3** (Agent impersonation)**.** $\mathcal{A}$ impersonates an agent $A$ if $\mathcal{A}$ completes a ZAP handshake with an honest peer $T$ such that $T$ accepts the session as authenticated to $A$ and $\mathcal{A}$ does not hold $A$'s secret signing key.

**Definition 4** (Cross-session replay)**.** A replay event occurs when $\mathcal{A}$ injects into an honest session $sid_2$ a frame originally accepted in a distinct session $sid_1$, and the recipient accepts it as fresh.

### 6.2 Theorems

**Theorem 5** (Non-repudiation)**.** *For any PPT adversary $\mathcal{A}$ against the agent non-repudiation game of ZAP-MCP or ZAP-A2A,*

$$\mathsf{Adv}^{nonrep}_{\textit{ZAP-MCP}}(\mathcal{A}) \;\le\; q_s \cdot \mathsf{Adv}^{eufcma}_{\textit{Sig},\textit{ML-DSA-65}}(\mathcal{B}) + q_h \cdot 2^{-256}$$

*where $q_s$ is the number of signing queries, $q_h$ the number of hash queries, and $\mathcal{B}$ a constructed forger against ML-DSA-65 with running time within an additive constant of $\mathcal{A}$.*

**Theorem 6** (Agent impersonation infeasibility)**.** *For any PPT adversary $\mathcal{A}$ against ZAP-MCP/ZAP-A2A agent-impersonation,*

$$\mathsf{Adv}^{imp}_{\textit{ZAP}}(\mathcal{A}) \;\le\; \mathsf{Adv}^{indcca}_{\textit{KEM},\textit{ML-KEM-768}}(\mathcal{B}_1) + \mathsf{Adv}^{eufcma}_{\textit{Sig},\textit{ML-DSA-65}}(\mathcal{B}_2) + \frac{q_h^2}{2^{256}}.$$

**Lemma 7** (Replay freshness)**.** *For session nonces of length $\nu$ bits and per-session frame counts $\leq Q$,*

$$\Pr[\textit{replay accepted}] \;\leq\; \frac{Q(Q-1)}{2^{\nu+1}} + \textit{negl}.$$

*With $\nu = 96$ and $Q \leq 2^{40}$ this is $\leq 2^{-17}$, well below any operationally relevant threshold.*

*Remark* 8. Theorems 5 and 6 together imply that an agent fabric built on ZAP-MCP/ZAP-A2A delivers non-repudiable provenance and bilateral authentication under standard post-quantum assumptions. Lemma 7 forces $Q \cdot \nu$ budgeting in operational deployments; in practice this is satisfied trivially by 96-bit nonces.

# 7 Wire Performance

## 7.1 Baseline accounting

Consider a representative MCP `tools/call` request: $\rho$ encodes to roughly 192 bytes of JSON. The HTTP+SSE+JWT baseline adds the following overhead per request/response pair:

| **Layer** | **Bytes** |
|---|---|
| HTTP request line | 24 |
| HTTP host header | 32 |
| HTTP `Authorization` (JWT, RS256) | 920 |
| HTTP content-type / accept / user-agent / ... | 180 |
| HTTP/2 HPACK savings (estimated) | −200 |
| TLS record framing per record | 29 |
| SSE `event:`/`data:` framing | 22 |
| HTTP/2 frame headers | 9 |
| **HTTP+SSE+JWT total per call (one direction)** | **≈1016** |

A second JWT travels with the response only when callbacks reauthenticate; typical SSE keeps the original JWT context, so we do not count it on the return path. We also do not count TCP/IP framing, which is identical on both sides.

## 7.2 ZAP accounting

A ZAP frame carrying a signed MCP JSON-RPC request consists of:

| **Field** | **Bytes** |
|---|---|
| Frame magic + version | 4 |
| Sequence number | 8 |
| AEAD nonce (derived; on-wire nonce) | 12 |
| AEAD tag | 16 |
| Length prefix | 4 |
| Routing recipient ID (ZAP-RNS hash) | 16 |
| ML-DSA-65 signature (in plaintext, AEAD-protected) | 3309 |
| **ZAP per-frame fixed overhead** | **60 (excl. signature)** |
| **ZAP with per-frame ML-DSA-65 sig** | **3369** |

The signature is dominated by ML-DSA-65's 3309-byte size. For non-financial calls where third-party verifiability is not required, an operator may select *session-only* signatures (transcript signed once at handshake, frames carry only AEAD tags); fixed per-frame overhead is then 60 bytes versus HTTP+SSE+JWT's ≈1016 bytes — a 17× reduction. With per-frame ML-DSA-65

signatures the wire is heavier than HTTP+JWT, but JWT carries no third-party verifiability: the comparison is between a 3309-byte cryptographic record of intent versus a 920-byte bearer assertion.

### 7.3 Practical recommendation

For MCP tool calls of consequence (settlement, mutation, identity issuance), per-frame ML-DSA-65 signatures are mandatory and the cost is amortised over the audit value. For idempotent read-only queries, session-signed frames are sufficient, and ZAP-MCP is on the wire shorter than HTTP+SSE+JWT.

| **Call type** | **HTTP+SSE+JWT (B)** | **ZAP-MCP (B)** |
|---|---|---|
| read-only (e.g. `tools/list`) | 1016 | 60 |
| mutating, audited | 1016 | 3369 |

## 8 Composition with ZAP-RNS

The full ZAP-MCP call sequence is:

1. agent looks up tool name $n$ in ZAP-RNS (cached locally with TTL),
2. agent verifies record signature chain against the publisher root,
3. agent extracts $kem\_pk_T$, $sig\_pk_T$, $caps_T$,
4. agent connects to $T$ at the network endpoint also published in the record (an IP, hostname, or libp2p multiaddr),
5. agent runs the ZAP 1-RTT handshake, receiving and verifying $T$'s signed transcript,
6. agent issues ZAP-MCP frames as in §4.

No HTTP layer participates. No JWT is ever issued. There is no *a priori* HTTPS endpoint. The identity of $T$ is its $kem\_pk_T$, not any name resolved by DNS.

## 9 Compliance Considerations

### 9.1 HIPAA Security Rule

The HIPAA Security Rule [25] requires "transmission security" (§164.312(e)) and "audit controls" (§164.312(b)). ZAP-MCP satisfies the encryption-in-transit requirement under hybrid ML-KEM-768 + X25519, and audit controls map to the per-frame signed log: a clinical agent's request to read a record is a third-party verifiable artifact, retainable for the statutory six years.

### 9.2 SOC 2

SOC 2 [2] Common Criteria CC6.7 (transmission of information) and CC7.2 (system monitoring) are addressed identically. The distinguishing benefit is that ZAP-logged events are *independently* verifiable; SOC 2 auditors can replay the verification without trusting the operator's logging pipeline.

### 9.3 GDPR Article 30

Article 30 [8] requires "records of processing activities" kept by the controller and processor. A ZAP-wrapped agent fabric naturally produces such records: each tool call's request and response are signed, dated (via session-bound monotonic sequence), and bound to the agent and tool identity. This is materially stronger than the typical Article 30 "description of categories of processing" prose, which is non-cryptographic.

### 9.4 Long-term retention and the quantum horizon

Medical, legal, and financial records routinely have retention periods of 6–75 years. Any TLS-only protection of those records today must be modelled under harvest-now-decrypt-later [18]: an attacker who records the TLS handshake in 2026 and obtains a CRQC by, say, 2035, recovers all session content. Hybrid ML-KEM-768 keying eliminates this class of risk for any traffic encrypted under ZAP.

## 10 Deployment Notes

### 10.1 Existing MCP servers

A backwards-compatible adapter pattern is straightforward: a thin ZAP-MCP sidecar terminates frames, validates the agent identity and capability claims, then forwards plain JSON-RPC over a UNIX domain socket to the existing MCP server process. The sidecar adds ≈200 lines and runs in process; this is the recommended migration path for tool authors.

### 10.2 Existing A2A agents

A2A agents accept agent cards by URL. ZAP-A2A extends this with ZAP-RNS-resolved cards. Agents that wish to interoperate with both can publish a A2A-card-by-URL whose contents are byte-for-byte the canonical CBOR encoding of the ZAP-RNS record content; verifiers can then cross-check.

### 10.3 Key management

Long-term ML-DSA-65 signing keys are held in HSMs or platform secure enclaves. Per-session handshake keys are derived from the long-term keys plus ephemeral KEM material; session keys are zeroised at session close. Backup of long-term keys is by Shamir secret sharing across operator-distinct custodians; in regulated environments a regulator may hold one share with auditable retrieval policy.

## 11 Gap Items and Future Work

We are explicit about what ZAP-MCP/ZAP-A2A does not yet solve.

1. **Revocation distribution.** If a long-term agent signing key is compromised, every cached ZAP-RNS record pointing at it must be poisoned. We currently rely on short ZAP-RNS TTLs (default 600 seconds) plus a transparency-log-style revocation feed. A formal CRL-equivalent distribution scheme suitable for a fleet of $10^6$ agents is open work. The trade-off space is short TTLs (chatty resolvers) versus signed delta updates over a gossip overlay (complex, risk of gossip-poisoning).
2. **Capability claim semantics.** ZAP-RNS records carry a capability vector, but the schema is application-specific. A canonical ontology for tool capabilities (read-only, idempotent,

mutating, financial-impact) would let policy be expressed transport-side rather than per-application.

3. **Indirect prompt injection.** ZAP-MCP attributes tool output to a tool, but does not detect when that output is a prompt-injection payload. Mitigation belongs at the model integration layer; ZAP provides forensic attribution.
4. **Quantum-safe ZAP-RNS root.** The ZAP-RNS publisher root key itself must be ML-DSA-65 (or SLH-DSA-256s for ultra-long horizon). A governance procedure for rotating that root in a federated agent fabric is open.
5. **Tamarin/ProVerif analysis.** We have proven properties in the computational model (Section 6). A symbolic model verification along the lines of [7] for full ZAP-MCP session establishment is in progress.
6. **Side-channel resistance of agent-side ML-DSA-65.** On phones and laptops running an LLM agent, ML-DSA-65 signing must run in constant time; the reference implementation does, but operator-grade deployments should be verified against the target microarchitecture.

## 12 Related Work

**TLS-style mutual authentication.** TLS [22] with client certificates achieves mutual authentication but is bound to hostnames and X.509 PKIs [6]; agent populations are fundamentally not hostname-shaped. ACME [3] and short-lived certificates partially address dynamism but inherit the X.509 trust model.

**DIDs and Verifiable Credentials.** W3C DIDs [21, 26, 28] and Verifiable Credentials [27] provide a self-sovereign identity model close in spirit to ZAP-RNS. ZAP-RNS differs by enforcing that resolution yields a KEM public key and binds transport to it; this collapses the "identity proof" and "transport key" into a single artifact and removes a class of failure modes where DID resolution and transport authentication disagree.

**COSE/JOSE signed envelopes.** COSE [23] and JWE [13] provide signed and encrypted message envelopes. They do not specify a session establishment, do not pin transport identity to KEM keys, and do not provide replay freshness binding. ZAP uses COSE-style canonicalisation under the covers but adds the session and identity layers.

**Agent governance literature.** [24] articulates non-cryptographic governance practices for agentic AI. Cryptographic audit substrate as offered by ZAP-MCP/ZAP-A2A is complementary and provides the substrate on which governance practices can be enforced.

## 13 Conclusion

The agent fabric is the new universal joint of computation. Its security cannot be the security of HTTP plus a bearer token. We have shown that wrapping MCP and A2A in ZAP delivers post-quantum confidentiality, mutual authentication of agents and tools by KEM identity, non-repudiation of every message via per-frame ML-DSA-65 signatures, and provenance suitable for HIPAA, SOC 2, and GDPR. The construction is shorter on the wire than the HTTP+SSE+JWT baseline for read-only calls and competitive for audit-grade mutating calls. Open work centres on revocation distribution at fleet scale, canonical capability ontologies, and symbolic model verification of the full handshake. We invite implementors of agent platforms to treat this work as a starting specification.

## References


[1] Agent2Agent Protocol Working Group. Agent2agent (A2A) protocol specification. https://a2a-protocol.org/specification, 2025. Agent cards, tasks, artifacts; HTTP/JSON transport.

[2] AICPA. Trust services criteria for security, availability, processing integrity, confidentiality, and privacy (SOC 2), 2017.

[3] Richard Barnes, Jacob Hoffman-Andrews, Daniel McCarney, and James Kasten. RFC 8555: Automatic certificate management environment (ACME), 2019.

[4] Daniel J. Bernstein, Niels Duif, Tanja Lange, Peter Schwabe, and Bo-Yin Yang. High-speed high-security signatures. Journal of Cryptographic Engineering, 2(2):77–89, 2011.

[5] Joppe W. Bos, Léo Ducas, Eike Kiltz, Tancrède Lepoint, Vadim Lyubashevsky, John M. Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS-Kyber: A CCA-secure module-lattice-based KEM. In *IEEE European Symposium on Security and Privacy, EuroS&P*, pages 353–367, 2018.

[6] David Cooper, Stefan Santesson, Stephen Farrell, Sharon Boeyen, Russell Housley, and Tim Polk. RFC 5280: Internet X.509 public key infrastructure certificate and CRL profile, 2008.

[7] Cas Cremers, Marko Horvat, Jonathan Hoyland, Sam Scott, and Thyla van der Merwe. A comprehensive symbolic analysis of TLS 1.3. In *ACM CCS*, pages 1773–1788, 2017.

[8] European Parliament and Council. Regulation (EU) 2016/679 (GDPR), article 30: Records of processing activities, 2016.

[9] Ian Fette and Alexey Melnikov. RFC 6455: The WebSocket protocol, 2011.

[10] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023.

[11] Dick Hardt. RFC 6749: The OAuth 2.0 authorization framework, 2012.

[12] Michael B. Jones, John Bradley, and Nat Sakimura. RFC 7519: JSON web token (JWT), 2015.

[13] Michael B. Jones and Joe Hildebrand. RFC 7516: JSON web encryption (JWE), 2015.

[14] JSON-RPC Working Group. JSON-RPC 2.0 specification. https://www.jsonrpc.org/specification, 2013.

[15] Hugo Krawczyk. SIGMA: The SIGn-and-MAc approach to authenticated Diffie-Hellman and its use in the IKE protocols. In *CRYPTO*, volume 2729 of *LNCS*, pages 400–425, 2003.

[16] Hugo Krawczyk. Cryptographic extraction and key derivation: The HKDF scheme. In *CRYPTO*, volume 6223 of *LNCS*, pages 631–648, 2010.

[17] Model Context Protocol. Model context protocol specification. https://modelcontextprotocol.io/specification, 2024. JSON-RPC 2.0 transport over stdio, HTTP+SSE, and WebSocket.

[18] Michele Mosca. Cybersecurity in an era with quantum computers: Will we be ready? IEEE Security & Privacy, 2018.

[19] NIST. FIPS 203: Module-lattice-based key-encapsulation mechanism standard, 2024.

[20] NIST. FIPS 204: Module-lattice-based digital signature standard, 2024.

[21] Michael Prorock, Orie Steele, and Manu Sporny. did:web method specification, 2024.

[22] Eric Rescorla. RFC 8446: The transport layer security (TLS) protocol version 1.3, 2018.

[23] Jim Schaad. RFC 9052: CBOR object signing and encryption (COSE), 2022.

[24] Yonadav Shavit, Sandhini Agarwal, Miles Brundage, Steven Adler, Cullen O'Keefe, Rosie Campbell, Teddy Lee, Pamela Mishkin, Tyna Eloundou, Alan Hickey, Katarina Slama, Lama Ahmad, Paul McMillan, Alex Beutel, Alexandre Passos, and David G. Robinson. Practices for governing agentic AI systems. In *OpenAI Research*, 2024.

[25] U.S. Department of Health and Human Services. HIPAA security rule, 45 CFR part 164, subpart C, 2003.

[26] W3C. Decentralized identifiers (DIDs) v1.0. W3C Recommendation, 2022.

[27] W3C. Verifiable credentials data model v2.0. W3C Recommendation Candidate, 2024.

[28] Dmitri Zagidulin, Manu Sporny, and Dave Longley. did:key method specification, 2022.

[29] ZAP Protocol Authors. ZAP: A post-quantum, authenticated, zero-copy transport. ZAP Protocol Working Notes, 2025.

[30] ZAP Protocol Authors. ZAP-RNS: Identity-bound resource naming for post-quantum transports. ZAP Protocol Working Notes, 2025.

[31] ZAP Protocol Authors. ZAP-MCP and ZAP-A2A: Formal security proofs (companion proof document). https://zap-proto.io/proofs/agent-communication, 2026.